

Java Intern Mini-Projects Pack (Level Up) — 20 Assignments

A step up in difficulty: stronger OOP design, data structures, file persistence, testing, and light concurrency. Most are CLI-first; optional stretch goals can move toward real-world app patterns.

Submission Checklist

- Source code (Git repo link or zip) with clean structure (src/, test/, resources/ if used).
- README.md: how to run, sample commands, expected outputs, assumptions, edge cases.
- At least 3-5 unit tests for non-trivial logic (where applicable).
- Short design note: classes/modules overview + why you chose the approach.

Suggested Rubric (100 points)

- Correctness (35): meets requirements; stable behavior.
- Design & structure (20): good separation of concerns; sensible classes/interfaces.
- Validation & edge cases (15): robust input handling; no crashes.
- Testing (15): meaningful unit tests; clear coverage of rules.
- User experience (10): helpful prompts, consistent commands, readable output.
- README & run instructions (5): easy to run and verify.
- Bonus (+10): stretch features done well (performance, persistence, concurrency, etc.).

Starter Patterns (Recommended)

- Use a Command loop (parse input; route to handlers).
- Model objects (POJOs) + service layer (business rules) + storage layer (file).
- Prefer interfaces for pluggable parts (Storage, PricingStrategy, etc.).
- Write small pure methods to make testing easy.

CLI Command Loop Skeleton

```
while (true) {
    System.out.print("> ");
    String line = sc.nextLine().trim();
    if (line.equalsIgnoreCase("exit")) break;
    if (line.isBlank()) continue;

    String[] parts = line.split("\\s+", 2);
    String cmd = parts[0].toLowerCase();
    String args = parts.length > 1 ? parts[1] : "";

    switch (cmd) {
        case "help" -> printHelp();
        default -> System.out.println("Unknown command. Type 'help'.");
    }
}
```

}

Assignments (20) — Level Up

Each assignment includes objective, steps, sample I/O, and stretch ideas.

1) Library Management CLI (OOP + Persistence)

Objective: Manage books/members/loans with clean OOP and save/load to file.

What to do

- 1 Design classes: Book(isbn,title,author), Member(id,name), Loan(bookIsbn, memberId, dueDate).
- 2 Commands: add-book, add-member, loan, return, list-books, list-loans, find-book <keyword>.
- 3 Enforce rules: can't loan an already-loaned book; validate IDs; due date required.
- 4 Persist state to disk (CSV or simple JSON-like format) on 'save' and load on startup.
- 5 Write unit tests for loan/return rules.

Sample Input

```
add-book 9780134685991 Effective_Java Bloch
add-member M1 Alice
loan 9780134685991 M1 2026-06-01
list-loans
return 9780134685991
list-loans
exit
```

Sample Output

```
Added book: 9780134685991
Added member: M1
Loan created. Due: 2026-06-01
Loans: [9780134685991 -> M1]
Returned: 9780134685991
Loans: (none)
```

Stretch (Bonus) Ideas

- Overdue report
- Export inventory to CSV
- Pluggable storage (interface Storage)

2) Personal Finance Ledger (CSV Import/Export)

Objective: Track transactions, categories, and monthly summaries with CSV I/O.

What to do

- 1 Transaction: date, description, category, amount (+income, -expense).
- 2 Commands: add, list, summary <YYYY-MM>, category-summary <YYYY-MM>, import <file>, export <file>.
- 3 Validation: date format, amount numeric, category non-empty.

- 4 Compute totals: income, expenses, net, top categories.
- 5 Add tests for summary math and parsing.

Sample Input

```
add 2026-05-01 Paycheck income 2500
add 2026-05-02 Grocery food -54.20
summary 2026-05
export ledger.csv
exit
```

Sample Output

```
May 2026: income=2500.00 expenses=54.20 net=2445.80
Exported: ledger.csv
```

Stretch (Bonus) Ideas

- Budget limits with alerts
- Recurring transactions
- Pretty table output

3) LRU Cache Implementation (Data Structures)

Objective: Implement an LRU cache with $O(1)$ get/put.

What to do

- 1 Build `LRUCache<K,V>` with capacity.
- 2 Use `HashMap` for lookup and doubly linked list for recency ordering.
- 3 Commands to demo: capacity `<n>`, put `<k> <v>`, get `<k>`, dump.
- 4 Write tests verifying eviction order.

Sample Input

```
capacity 2
put A 1
put B 2
get A
put C 3
dump
exit
```

Sample Output

```
OK
OK
1
Evicted: B
Cache: A=1, C=3
```

Stretch (Bonus) Ideas

- Make it thread-safe (synchronized/locks)

- Add stats: hits/misses

4) Task Scheduler with Priorities (PriorityQueue)

Objective: Schedule tasks by priority and due time.

What to do

- 1 Task: id, title, priority(1-5), dueDateTime.
- 2 Commands: add-task, next, complete <id>, list, overdue.
- 3 Use PriorityQueue ordering: earliest due first, tie-break higher priority.
- 4 Test comparator ordering.

Sample Input

```
add-task T1 Deploy 5 2026-06-01T10:00
add-task T2 Docs 2 2026-05-29T09:00
next
complete T2
next
exit
```

Sample Output

```
Next: T2 Docs (2) 2026-05-29T09:00
Completed: T2
Next: T1 Deploy (5) 2026-06-01T10:00
```

Stretch (Bonus) Ideas

- Persist tasks
- Reschedule command
- Recurring tasks

5) Mini Search Engine (Inverted Index)

Objective: Index a folder of text files and support keyword search.

What to do

- 1 Read all .txt files from a directory.
- 2 Build inverted index: word -> map(filename -> count).
- 3 Commands: index <dir>, search <word>, top <n> <word>, stats.
- 4 Normalize text: lowercase, strip punctuation.
- 5 Add tests for tokenization and index merge.

Sample Input

```
index ./docs
search java
top 3 java
exit
```

Sample Output

Indexed 12 files. Unique words: 1845
Found in: file1.txt(4), file7.txt(1)
Top: file1.txt(4), file7.txt(1)

Stretch (Bonus) Ideas

- Multi-word AND/OR queries
- Phrase search (simple)
- Persist index to disk

6) Markdown-to-HTML Lite Converter

Objective: Convert basic Markdown syntax to HTML.

What to do

- 1 Support: headings (#, ##), bold (**text**), italics (*text*), code (`code`), bullet lists (- item).
- 2 Read from input file and write to output.html.
- 3 Keep it simple: line-by-line parsing plus inline replacements.
- 4 Add tests for each syntax transformation.

Sample Input

```
convert input.md output.html  
exit
```

Sample Output

Converted: input.md -> output.html

Stretch (Bonus) Ideas

- Support links [text](url)
- Support fenced code blocks
- Generate a table of contents

7) Expression Evaluator (Shunting-yard)

Objective: Evaluate math expressions with precedence and parentheses.

What to do

- 1 Accept expressions like: $3 + 4 * (2 - 1)$.
- 2 Tokenize input; convert infix to postfix (RPN) using stacks.
- 3 Evaluate postfix.
- 4 Handle whitespace and invalid tokens gracefully.
- 5 Write tests for precedence and parentheses.

Sample Input

```
eval 3 + 4 * (2 - 1)
eval (10 / 2) + 7
exit
```

Sample Output

Result: 7
Result: 12

Stretch (Bonus) Ideas

- Add exponent operator ^
- Support unary minus
- Add variables (let x=3)

8) Sudoku Validator + Solver (Backtracking)

Objective: Validate a Sudoku board and optionally solve it.

What to do

- 1 Read a 9x9 grid from a file (0 for empty).
- 2 Validate rows, columns, and 3x3 boxes.
- 3 If valid and has empties, solve with backtracking and print solution.
- 4 Add tests for validator logic.

Sample Input

```
load sudoku.txt
validate
solve
exit
```

Sample Output

Board valid: YES
Solved. Steps: 241
(prints solved grid)

Stretch (Bonus) Ideas

- Count multiple solutions
- Choose cell with fewest candidates

9) Maze Generator + Solver (BFS/DFS)

Objective: Generate a random maze and find a path from start to end.

What to do

- 1 Generate a grid maze using randomized DFS (carve passages).
- 2 Solve using BFS for shortest path.
- 3 Print maze and solution path in ASCII.

4 Tests: BFS returns a valid path for small fixed mazes.

Sample Input

```
generate 15 10
solve
exit
```

Sample Output

```
Maze 15x10 generated.
Path length: 32
(print maze with solution)
```

Stretch (Bonus) Ideas

- Let user place walls
- Compare DFS vs BFS path lengths

10) File Organizer (Rules Engine)

Objective: Organize files into folders based on configurable rules.

What to do

- 1 Read a directory and classify by extension (jpg->Images, pdf->Docs, etc.).
- 2 Support a rules config file (e.g., rules.properties).
- 3 Commands: scan <dir>, plan, apply, undo.
- 4 Keep an operation log to undo moves.
- 5 Tests: rule parsing + plan generation.

Sample Input

```
scan ./downloads
plan
apply
exit
```

Sample Output

```
Planned moves: 23
Applied moves: 23
```

Stretch (Bonus) Ideas

- Duplicate detection by hash
- Dry-run mode
- Custom ignore patterns

11) URL Shortener (In-memory + Persistence)

Objective: Create short codes for long URLs and resolve them.

What to do

- 1 Commands: shorten <url>, resolve <code>, list, save, load.
- 2 Generate code using base62 of an incrementing id or hash.
- 3 Prevent duplicates: same URL returns existing code.
- 4 Persist mappings to a file.
- 5 Tests: code generation and resolve behavior.

Sample Input

```
shorten https://example.com/very/long
resolve aZ3
list
exit
```

Sample Output

```
Created: aZ3
https://example.com/very/long
Mappings: aZ3 -> https://example.com/very/long
```

Stretch (Bonus) Ideas

- Expiration dates
- Click counts
- Custom aliases

12) Movie Recommender (Streams + Ranking)

Objective: Recommend items using aggregation and simple similarity.

What to do

- 1 Load ratings CSV: userId,movieId,rating.
- 2 Compute average rating per movie and top-N list.
- 3 Implement similar users by cosine similarity on common ratings (simple).
- 4 Commands: top <n>, recommend <userId> <n>.
- 5 Use Java Streams where helpful; tests for aggregations.

Sample Input

```
load ratings.csv
top 5
recommend U12 5
exit
```

Sample Output

```
Top 5: M3(4.7), M8(4.6), ...
Recommendations for U12: M9, M2, ...
```

Stretch (Bonus) Ideas

- Filter by genre (extra file)

- Cold-start handling

13) Multi-threaded Log Processor (Concurrency)

Objective: Process a large log file using multiple threads and aggregate metrics.

What to do

- 1 Split a log file into chunks (by lines) and process in worker threads.
- 2 Compute counts per level, top endpoints, top error messages.
- 3 Merge partial results safely (ConcurrentHashMap or synchronized merge).
- 4 Command: analyze <file> <threads>.
- 5 Tests: merging logic with small sample.

Sample Input

```
analyze app.log 4
exit
```

Sample Output

```
Threads: 4
INFO=1203 WARN=88 ERROR=17
Top endpoints: /login(220), /transfer(140)
```

Stretch (Bonus) Ideas

- Measure time vs single-thread
- Add percentile latency if present

14) Producer-Consumer Simulation (BlockingQueue)

Objective: Simulate order processing with producers and consumers.

What to do

- 1 Producer threads create orders at random intervals.
- 2 Consumer threads process orders and update stats.
- 3 Use BlockingQueue with bounded capacity.
- 4 Commands: start <producers> <consumers> <seconds>, stats.
- 5 Tests: stats calculations in deterministic mode.

Sample Input

```
start 2 3 10
stats
exit
```

Sample Output

```
Ran 10s. Produced=180 Consumed=175 Dropped=0 AvgWaitMs=12
```

Stretch (Bonus) Ideas

- Backpressure behavior
- Graceful shutdown
- Per-consumer throughput

15) TCP Chat (Client + Server)

Objective: Build a simple multi-client chat using sockets.

What to do

- 1 Create a server that accepts multiple clients (thread per client or thread pool).
- 2 Broadcast messages to all connected clients.
- 3 Support commands: /name <nick>, /who, /quit.
- 4 Handle disconnects gracefully.
- 5 Tests: message formatting utilities.

Sample Input

```
(Server) start 5555
(Client1) /name alice
(Client2) /name bob
(Client1) hello
(Client2) hi
(Client2) /quit
```

Sample Output

```
[alice]: hello
[bob]: hi
User bob left.
```

Stretch (Bonus) Ideas

- Private messages: /msg <user> <text>
- Message history for new joiners

16) REST Client CLI (HttpClient + Retry)

Objective: Call a REST endpoint, parse JSON minimally, and implement retries.

What to do

- 1 Use `java.net.http.HttpClient` to call a given URL.
- 2 Support GET with optional headers.
- 3 Parse JSON minimally (string-based) OR allow a small JSON library via Maven.
- 4 Implement retry with exponential backoff for 5xx/timeouts.
- 5 Commands: `get <url>`, `get <url> --header k:v`.

Sample Input

```
get https://api.example.com/status
exit
```

Sample Output

```
200 OK
{...}
Parsed field: status=UP
```

Stretch (Bonus) Ideas

- Add POST with body
- Config file for endpoints
- Circuit breaker (simple)

17) Config Loader + Validator (Schema-lite)

Objective: Load app config from a file and validate required fields/types.

What to do

- 1 Define config fields: appName(string), port(int), features(list), timeoutMs(int).
- 2 Load from properties or JSON; validate missing/invalid values.
- 3 Print a friendly validation report.
- 4 Add tests for validation rules.

Sample Input

```
load config.properties
validate
exit
```

Sample Output

```
Config valid: NO
- port must be 1024-65535
- timeoutMs missing
```

Stretch (Bonus) Ideas

- Defaulting rules
- Environment overrides
- Generate sample config

18) Plugin-based Calculator (Interfaces + ServiceLoader)

Objective: Discover operations at runtime and execute them.

What to do

- 1 Define interface Operation { String name(); double apply(double a, double b); }.

- 2 Implement operations (add/sub/mul/div) as separate classes.
- 3 Load operations dynamically (simple: register list; advanced: ServiceLoader).
- 4 CLI: ops, calc <op> <a> .
- 5 Tests for each operation.

Sample Input

```
ops
calc add 2 3
calc mul 4 5
exit
```

Sample Output

```
Available: add, sub, mul, div
Result: 5
Result: 20
```

Stretch (Bonus) Ideas

- Add mod/pow
- Allow unary ops
- Load plugins from external jars (advanced)

19) Design Patterns: Pricing Engine (Strategy)

Objective: Implement a pricing engine with swappable discount strategies.

What to do

- 1 Cart items with price and quantity.
- 2 Strategies: NoDiscount, PercentageOff, BuyOneGetOne, TieredDiscount.
- 3 CLI: add-item, set-discount <type> <params>, total.
- 4 Write tests asserting totals per strategy.

Sample Input

```
add-item coffee 3.50 2
set-discount percent 10
total
exit
```

Sample Output

```
Subtotal: 7.00
Discount: 0.70
Total: 6.30
```

Stretch (Bonus) Ideas

- Combine discounts safely
- Receipt printing with line-level discounts

20) Mini Version Control (Hashing + Snapshots)

Objective: Track file snapshots with commits and restore versions.

What to do

- 1 Commands: init, add <file>, commit <msg>, log, checkout <commitId>.
- 2 Store snapshots in a hidden folder (e.g., .minivcs/) by content hash.
- 3 Commit records: id, timestamp, message, list of tracked file hashes.
- 4 Tests for hashing and commit metadata.

Sample Input

```
init
add notes.txt
commit "first"
log
checkout 1
exit
```

Sample Output

```
Initialized repository
Staged: notes.txt
Committed: 1 first
1 first (2026-05-21T12:00)
Checked out commit 1
```

Stretch (Bonus) Ideas

- Status command
- Branch-lite (named pointers)
- Diff (line-by-line)